

RELATÓRIO: PROGRAMAÇÃO ORIENTADA A OBJETOS EM C++

1. INTRODUÇÃO E CONTEXTO HISTÓRICO

A linguagem C++ surgiu da necessidade de Bjarne Stroustrup, no **Bell Labs (AT&T)** em Nova Jersey, de criar uma ferramenta que suportasse a complexidade de sistemas distribuídos. A visão do criador para o projeto era clara: *"Eu queria construir um sistema multi-computador com um sistema de comunicação"*.

A grande inovação foi a **Síntese de Paradigmas**: o C++ foi projetado para ser uma evolução do C, integrando os conceitos de classes da linguagem Simula em uma base de alta performance. Essa estrutura é dividida em duas camadas fundamentais:

- **Camada de Abstração (Simula)**: Fornece a organização de sistemas complexos via objetos.
- **Camada de Eficiência (C)**: Garante a execução próxima ao hardware e alta velocidade.

CRONOLOGIA DE EVOLUÇÃO E PADRONIZAÇÃO		
Ano	Marco Histórico	Impacto na Linguagem
1979	"C with Classes"	Início do projeto unindo a eficiência do C com a organização do Simula.
1983	O Batismo (C++)	Rick Mascitti sugere o nome usando o operador de incremento (evolução do C).
1985	Lançamento Comercial	Publicação do livro de referência e do compilador <i>Cfront</i> .
1989	Comitê de Padronização	Início dos esforços conjuntos ANSI e ISO para criar um padrão universal.
1998	ISO C++98	Ratificação do primeiro padrão internacional oficial da linguagem.

Essa evolução é regida pelo **Zero-overhead principle** (princípio do custo zero), que garante que o programador possa utilizar os pilares da POO sem sofrer perdas de desempenho desnecessárias, mantendo a eficiência de execução exigida por sistemas críticos.

2. APLICABILIDADE

Devido ao seu alto desempenho e controle refinado de memória, o C++ é a escolha predominante em cenários críticos. **Na infraestrutura global, a linguagem é usada na base de Sistemas Operacionais (núcleos do Windows, macOS e Linux), Finanças de Alta Frequência (sistemas de *trading* que operam em microssegundos) e Sistemas Embarcados (carros autônomos e dispositivos médicos).** A análise técnica de seu vasto ecossistema revela como as ferramentas utilizam os pilares da Orientação a Objetos para resolver problemas complexos em diferentes indústrias.

DOMÍNIOS E ECOSSISTEMA DE FRAMEWORKS		
Setor de Atuação	Frameworks Líderes	Características Técnicas e Uso da POO
Interfaces Gráficas (GUI)	Qt (v6.11) wxWidgets (v3.2.10)	O Qt é um framework multiplataforma completo. Ele exemplifica o pilar da Herança , onde todos os componentes visuais (botões, janelas) derivam de classes base robustas. O wxWidgets utiliza componentes nativos do SO, oferecendo alta estabilidade.
Motores de Jogos (Engines)	Unreal Engine (v5.7) Godot Engine (v4.6.2)	A Unreal Engine domina o mercado AAA combinando herança complexa em C++ com "Blueprints" (um sistema visual que encapsula código C++ em blocos lógicos). O Godot possui seu núcleo escrito em C++, demonstrando extrema flexibilidade arquitetural e forte Abstração .
Back-end Web e Redes	Boost.Asio Drogon (v1.9.12)	O Drogon é um framework focado em alto desempenho para APIs web. O Boost.Asio fornece a base para operações de I/O Assíncrono (capacidade de processar múltiplas requisições de rede simultaneamente sem travar o sistema), exigindo manipulação avançada de polimorfismo.

Machine Learning (IA)	TensorFlow (C++ API) PyTorch (LibTorch)	O processamento matemático pesado de redes neurais roda em C++ para garantir máxima performance no hardware (CPU/GPU). Eles utilizam o Encapsulamento estrito para esconder a complexidade dos cálculos das interfaces simplificadas em Python.
Persistência de Dados	ODB SOCI	O ODB é uma ferramenta ORM (Object-Relational Mapping). Na prática, ele aplica o encapsulamento para converter tabelas de bancos de dados relacionais diretamente em objetos (classes) C++. O SOCI faz um mapeamento similar, permitindo forte Abstração sobre diversos bancos (MySQL, PostgreSQL).
Qualidade e Testes	Google Test (gtest) Catch2 (v3.14.0)	Essenciais para a indústria, garantem que as classes funcionem corretamente. Utilizam Polimorfismo dinâmico massivamente. O Google Test oferece "fixtures" (ambientes isolados que preparam objetos C++ antes da execução dos testes).

3. MAPEAMENTO DOS PILARES: C++ VS C#

Embora o C# tenha herdado grande parte da sua sintaxe da família C, a implementação dos pilares da Orientação a Objetos diverge profundamente devido aos objetivos de cada ecossistema. O C++ foca no desempenho bruto e no controle manual do hardware, enquanto o C# prioriza a produtividade corporativa através da **CLR (Common Language Runtime)**, a máquina virtual do .NET que gerencia a execução e a segurança do código em segundo plano.

COMPARATIVO DE IMPLEMENTAÇÃO		
Pilar da POO	Implementação em C++	Implementação em C#
Abstração	Baseada em Funções Virtuais Puras (ex: virtual void func() = 0;). Não existe a palavra-chave estrita interface.	Utiliza palavras-chave explícitas nativas: interface e abstract class.
Encapsulamento	Modificadores funcionam como rótulos de bloco de escopo (ex: public:). Fomenta a criação estruturada de getters e setters manuais.	Modificadores são aplicados membro a membro. Utiliza Properties nativas ({ get; set; }) para simplificar o acesso.
Herança	Suporta Herança Múltipla de classes (permite derivar de várias classes simultaneamente).	Suporta apenas Herança Simples de classes, mas permite assinatura de múltiplas interfaces.
Polimorfismo	O despacho dinâmico exige o uso de ponteiros (*) ou referências (&) na classe base para funcionar em tempo de execução.	O despacho é gerenciado nativamente. O uso da palavra-chave override na classe filha é obrigatório.

3.1 Diferenças de Arquitetura e Execução

3.1.1 O Gerenciamento de Memória (Automático vs. Manual)

No C#, existe o **Garbage Collector**, um processo em segundo plano que identifica objetos sem referência e libera a memória automaticamente. Isso evita falhas, mas gera um custo de processamento imprevisível.

No C++, segue-se o princípio do **RAII (Resource Acquisition Is Initialization)**. A regra arquitetural define que o controle da memória é estritamente do programador. A alocação de memória exige a contrapartida da liberação através de um **Destrutor** (método invocado no fim da vida útil do objeto). Para modernizar e garantir segurança, o ecossistema C++ atual adota **Smart Pointers**, que gerenciam a destruição dos dados autonomamente ao saírem de escopo.

3.1.2 O Conflito da Herança (O Problema do Diamante)

Quando uma classe derivada herda de duas classes base que, por sua vez, compartilham a mesma superclasse (formando um losango hierárquico), o compilador sofre de ambiguidade estrutural. Isso é o **Problema do Diamante**.

O C# evita essa falha proibindo a herança múltipla de classes. Já o C++ permite a herança múltipla, mas soluciona a ambiguidade exigindo o uso da **Herança Virtual**, uma técnica que garante em tempo de compilação que apenas uma instância da superclasse original seja mantida na memória.

3.1.3 Velocidade vs Flexibilidade (O uso da VTable)

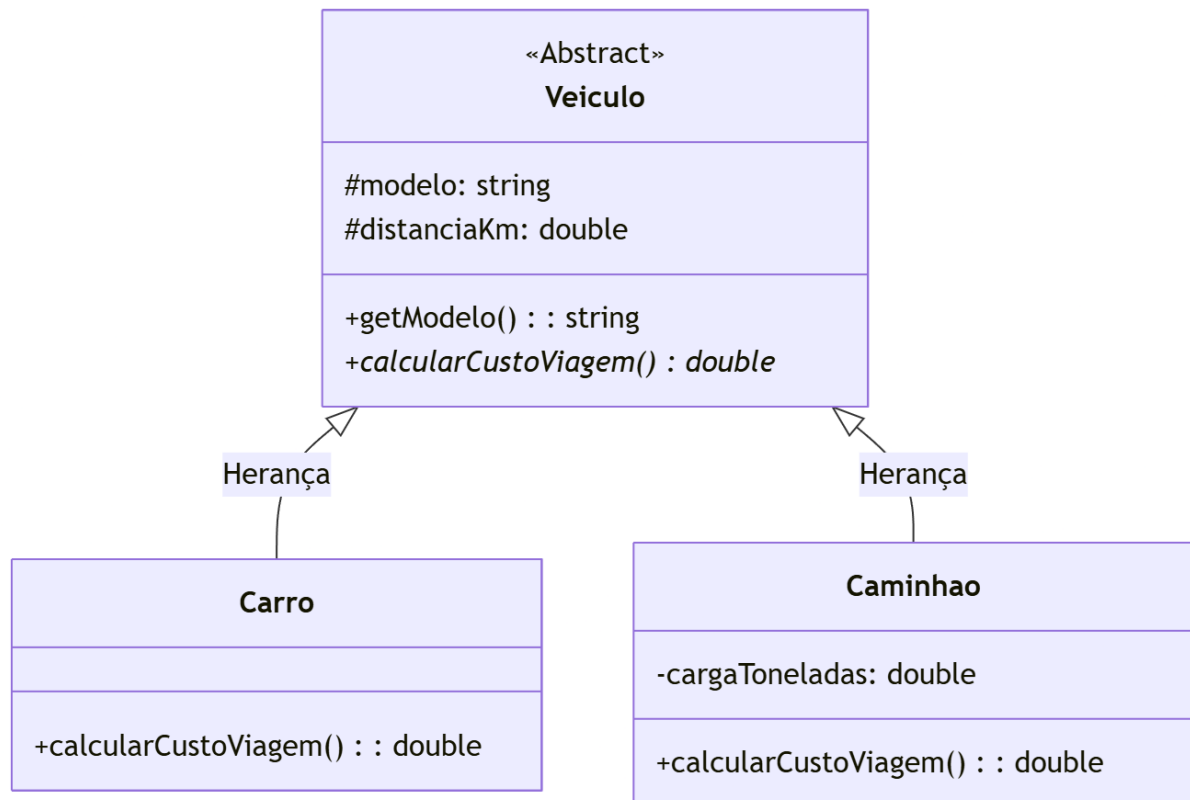
O C++ é altamente veloz pois prioriza a resolução de chamadas durante a compilação (**Early Binding**). Contudo, o Polimorfismo exige decisões em tempo de execução. Para solucionar isso sem ferir a performance, o C++ implementa a **VTable (Tabela Virtual)**: uma matriz leve de ponteiros de função que o sistema consulta rapidamente para determinar qual método sobrescrito invocar.

No C#, esse processo de despacho dinâmico é inteiramente abstraído e gerenciado pela máquina virtual (CLR), resultando em um desenvolvimento mais seguro, porém com um custo computacional atrelado ao ambiente gerenciado.

4. EXEMPLO DE APLICAÇÃO

4.1 Modelagem do Sistema (Diagrama UML)

O sistema desenvolvido simula uma Gestão de Frota, onde diferentes tipos de transporte calculam os seus custos de forma independente. O diagrama abaixo representa o relacionamento "É-UM" (Herança) entre as entidades.



4.2 Código-Fonte (C++)

O código abaixo implementa a lógica de gestão de frota utilizando os pilares da Orientação a Objetos e seguindo boas práticas de organização.

```
#include <iostream>
#include <string>
#include <iomanip> // Para formatar as casas decimais

using namespace std;

// 1. ABSTRAÇÃO: Classe base que define o molde para qualquer veículo
```

```

class Veiculo
{
protected:
    string modelo;
    double distanciaKm;

public:
    Veiculo(string mod, double dist) : modelo(mod),
    distanciaKm(dist) {}

    // Destrutor Virtual: Garante a limpeza correta de memória das
    classes filhas
    virtual ~Veiculo() {}

    string getModelo() const
    {
        return modelo;
    }

    // Função Virtual Pura: Torna a classe abstrata
    virtual double calcularCustoViagem() = 0;
};

// 2. HERANÇA: Carro estende as funcionalidades de Veiculo
class Carro : public Veiculo
{
public:
    Carro(string mod, double dist) : Veiculo(mod, dist) {}

    // 3. POLIMORFISMO: Implementação específica para carros
    double calcularCustoViagem() override
    {
        return distanciaKm * 0.95; // Custo fixo por KM
    }
}

```

```

    }
};

// 2. HERANÇA: Caminhao estende Veiculo e adiciona novos atributos
class Caminhao : public Veiculo
{
private:
    double cargaToneladas;

public:
    Caminhao(string mod, double dist, double carga)
        : Veiculo(mod, dist), cargaToneladas(carga) {}

    // 3. POLIMORFISMO: Cálculo que considera o peso da carga
    double calcularCustoViagem() override
    {
        return (distanciaKm * 3.80) + (cargaToneladas * 60.0);
    }
};

// FUNÇÃO AUXILIAR PARA DEMONSTRAR O POLIMORFISMO
// Recebe um ponteiro para a classe base (Veiculo), mas executa o
código da classe filha.
void exibirRelatorio(Veiculo* v)
{
    cout << "Veiculo: " << v->getModelo()
        << " | Custo Total: R$ " << v->calcularCustoViagem() <<
endl;
}

int main()
{
    double distCarro, distCaminhao, pesoCarga;

```



```

cout << "### Sistema de Gestao de Frota P00 ###" << endl;

// ENTRADA DE DADOS
cout << "Distancia percorrida pelo Carro (km): ";
cin >> distCarro;

cout << "Distancia percorrida pelo Caminhao (km): ";
cin >> distCaminhao;
cout << "Peso da carga do Caminhao (toneladas): ";
cin >> pesoCarga;

// Instanciação manual dos objetos usando ponteiros
Veiculo* meuCarro = new Carro("Sedan Executivo", distCarro);
Veiculo* meuCaminhao = new Caminhao("Volvo FH", distCaminhao,
pesoCarga);

cout << fixed << setprecision(2);
cout << "\n--- Relatorio de Custos de Viagem ---" << endl;

// 4. APLICAÇÃO DO POLIMORFISMO
// A mesma função comporta-se de maneira diferente consoante o
objeto passado
exibirRelatorio(meuCarro);
exibirRelatorio(meuCaminhao);

// Libertação manual de memória (Gestão de Recursos em C++)
delete meuCarro;
delete meuCaminhao;
return 0;
}

```

4.3 Instruções para Execução

- **Recursos:** O sistema é executado de forma nativa pelo sistema operacional, não dependendo de máquinas virtuais externas.
- **Ferramentas:** Para compilar e rodar o projeto, recomenda-se a IDE **Code::Blocks** com o compilador **GCC/MinGW** integrado.
- **Procedimento:** Basta abrir o arquivo .cpp no Code::Blocks e utilizar o comando *Build and Run* (F9).

4.4 Explicação do Sistema, Entrada e Saída

- **Explicação:** O sistema solicita distância e carga no console e instancia dois veículos usando alocação dinâmica. A função `exibirRelatorio` trata ambos como `Veiculo`. Ao chamar `calcularCustoViagem()`, o método da classe filha correspondente é executado (polimorfismo). Ao final, a memória é liberada com `delete`.
- **Entrada Desejada:** Inserção manual de valores via teclado. Para este exemplo: 100 para a distância do carro, 100 para a distância do caminhão e 10 para o peso da carga.
- **Saída Esperada no Console:**

Sistema de Gestao de Frota POO

Distancia percorrida pelo Carro (km): 100

Distancia percorrida pelo Caminhao (km): 100

Peso da carga do Caminhao (toneladas): 10

--- Relatorio de Custos de Viagem ---

Veiculo: Sedan Executivo | Custo Total: R\$ 95.00

Veiculo: Volvo FH | Custo Total: R\$ 980.00

5. REFERÊNCIAS

1. [Stroustrup, Bjarne. *The Design and Evolution of C++*. Addison-Wesley, 1994.](#)
2. [Stroustrup, Bjarne. *The C++ Programming Language*. 4ª Edição. Addison-Wesley, 2013.](#)
3. [ISO/IEC 14882. *Programming languages — C++*. Organização Internacional de Normalização \(ISO\) e Comissão Eletrotécnica Internacional \(IEC\).](#)
4. [Lion, D. *Why JavaScript and Python are 8x and 29x slower than C++*. USENIX, 2022.](#)
5. [Chou, L. *A comparison of C++, Java and Python*. OAKTrust, Texas A&M University, 1997.](#)
6. [Fourment, M. *A comparison of common programming languages used in bioinformatics*. PMC, 2008.](#)